

# Week 6 Home Work

Submission Deadline: 29 May 2026, 11:59 PM

## 1. Bank Transaction History Tracker

Concepts: While Loop · Lists · String Methods · Conditionals · Data Types

### Problem Statement:

- A bank records every transaction a customer makes during a session.
- Each transaction has a type (deposit/withdraw) and an amount.
- The session starts with a balance of 100000 yen.
- Use a while loop — keep taking transactions until the customer types 'done'.
- For each transaction: store the type (uppercase) and amount in two separate lists.
- If withdraw and amount > current balance, reject it: print 'Rejected: Insufficient balance' and do NOT store it.
- Update balance after each valid transaction.
- After the session ends, print the full transaction history with running balance,
- the total number of deposits, total number of withdrawals, and closing balance.

### Acceptance Criteria:

- Start balance = 100000.
- Use while True with break when input == 'done'.
- Use .upper() on transaction type before storing.
- Use two lists: one for types, one for amounts — only store valid transactions.
- Use append() to add to each list.
- Use a counter for deposits and another for withdrawals.
- Use a while loop with index to print the history table.
- Use len() to confirm total transactions stored.

### Expected Output:

```
Opening Balance: 100000
Enter transaction type (deposit/withdraw or 'done'): deposit
Enter amount: 20000
Enter transaction type (deposit/withdraw or 'done'): withdraw
Enter amount: 50000
Enter transaction type (deposit/withdraw or 'done'): withdraw
Enter amount: 90000
Rejected: Insufficient balance
Enter transaction type (deposit/withdraw or 'done'): withdraw
Enter amount: 30000
Enter transaction type (deposit/withdraw or 'done'): done
=====
TRANSACTION HISTORY
=====
No. | Type | Amount | Balance
```

```
-----  
1 | DEPOSIT | +20000 | 120000  
2 | WITHDRAW | -50000 | 70000  
3 | WITHDRAW | -30000 | 40000  
-----
```

```
Total Deposits : 1  
Total Withdrawals: 2  
Closing Balance : 40000
```

## 2. Hospital Medicine Inventory System

Concepts: While Loop · Lists · Conditionals · Data Types · String Methods

### Problem Statement:

- A hospital pharmacy records incoming medicine stock every day.
- Each medicine entry has: name, quantity (int), and unit price (float).
- Use a while loop to keep accepting entries until the pharmacist types 'done'.
- Store medicine names (title case), quantities, and unit prices in three separate lists.
- After entry is complete, print the full inventory table.
- For each medicine, calculate total value (quantity x unit price).
- Mark stock status: quantity  $\geq 100 \rightarrow$  'Sufficient', quantity  $\geq 50 \rightarrow$  'Low', below 50  $\rightarrow$  'Critical'.
- Print the medicine with the highest total stock value at the end.

### Acceptance Criteria:

- Use while True with break when name == 'done'.
- Use .title() on medicine name before storing.
- Use int() for quantity and float() for unit price.
- Use three separate lists for names, quantities, and prices.
- Use append() to add to all three lists.
- Use a while loop with index to print the table and find max value.
- Use if/elif/else for stock status.
- Track highest value medicine using a variable.

### Expected Output:

```
Enter medicine name (or 'done'): paracetamol  
Enter quantity: 200  
Enter unit price: 15.5  
Enter medicine name (or 'done'): amoxicillin  
Enter quantity: 45  
Enter unit price: 80.0  
Enter medicine name (or 'done'): ibuprofen  
Enter quantity: 75  
Enter unit price: 25.0  
Enter medicine name (or 'done'): done
```

```
=====
PHARMACY INVENTORY REPORT
=====
Medicine | Qty | Unit Price | Total Value | Status
-----
Paracetamol | 200 | 15.5 | 3100.0 | Sufficient
Amoxicillin | 45 | 80.0 | 3600.0 | Critical
Ibuprofen | 75 | 25.0 | 1875.0 | Low
-----
Highest value stock: Amoxicillin (3600.0 yen)
```

---

### 3. ATM Withdrawal System

Concepts: While Loop · Conditionals · Data Types

#### Problem Statement:

- Set an initial account balance of 50000.
- Use a while loop to keep allowing withdrawals until the user types 0 to exit.
- Each iteration: ask for withdrawal amount.
- If amount > balance, print 'Insufficient funds'.
- If amount <= 0, exit the loop.
- Otherwise, deduct amount and print updated balance.
- After the loop, print the final balance.

#### Acceptance Criteria:

- Use a while True loop with a break condition.
- Use int() or float() to convert input.
- Use if/elif/else inside the loop.
- Print balance after every successful withdrawal.
- Print final balance after loop ends.

#### Expected Output:

```
Balance: 50000
Enter withdrawal amount (0 to exit): 10000
Withdrawal successful. Remaining balance: 40000
Enter withdrawal amount (0 to exit): 60000
Insufficient funds.
Enter withdrawal amount (0 to exit): 0
Thank you! Final balance: 40000
```

## 4. Supermarket Self-Checkout

Concepts: Lists · While Loop · Conditionals · Data Types

### Problem Statement:

- Use a while loop to keep adding products to a cart.
- Each iteration: ask for product name and price.
- If the user enters 'done' as the product name, stop the loop.
- Store each product and price in two separate lists.
- After the loop, print each item with its price.
- Print total number of items and the total bill amount.

### Acceptance Criteria:

- Use while True with a break when name == 'done'.
- Use two lists: one for names, one for prices.
- Use append() to add to both lists.
- Use float() to convert price.
- Use len() for item count and sum() for total bill.

### Expected Output:

```
Enter product (or 'done' to finish): apple
Enter price: 120
Enter product (or 'done' to finish): bread
Enter price: 85
Enter product (or 'done' to finish): done
--- Your Receipt ---
apple : 120.0
bread : 85.0
Total items: 2
Total bill: 205.0
```

---

## 5. Employee Attendance & Salary System (2D List)

Concepts: 2D Lists · While Loop · Conditionals · Data Types · String Methods

### Problem Statement:

- A company tracks employee attendance using a 2D list.
- Each row stores one employee's data: [name, days\_present, daily\_wage].
- The company has 4 employees — define the 2D list with pre-filled data.
- Use a while loop to process each employee and calculate their monthly salary.
- Deduct 10% from salary if days present is below 20.
- Add a 5% bonus if days present is 26 or more.
- Otherwise pay exact salary (days\_present x daily\_wage).
- Print a formatted payroll report for all employees.

### Acceptance Criteria:

- Define a 2D list with 4 rows, each as [name, days\_present, daily\_wage].

- Use a while loop with index to iterate through the list.
- Access values using records[i][0], records[i][1], records[i][2].
- Use if/elif/else for deduction, bonus, and standard pay.
- Use .title() when printing names.
- Print a clean payroll table with status (Deducted / Bonus / Standard).

#### Expected Output:

```
=====
MONTHLY PAYROLL REPORT
=====
Name | Days | Daily Wage | Salary | Status
-----
Aiko Yamamoto | 28 | 2000 | 58800 | Bonus
Kenji Mori | 18 | 1500 | 24300 | Deducted
Hana Sato | 22 | 1800 | 39600 | Standard
Riku Tanaka | 26 | 2500 | 68250 | Bonus
-----
Total Payroll: 190950
```

## 6. Library Book Search System

Concepts: Lists · While Loop · String Methods · Conditionals

#### Problem Statement:

- A library has a pre-defined list of book titles.
- Use a while loop to let the user search for books repeatedly.
- Each iteration: take a book title as input.
- If the user types 'exit', stop the loop.
- Search by converting both the input and list items to lowercase for comparison.
- Print 'Book found!' or 'Book not available' accordingly.
- Count and print total searches made after the loop ends.

#### Acceptance Criteria:

- Pre-define a list of at least 5 book titles.
- Use while True with a break on 'exit'.
- Use .lower() on both input and list item for comparison.
- Use a counter variable incremented each search.
- Print total search count after the loop.

#### Expected Output:

```
Enter book title (or 'exit' to quit): Python Crash Course
Book found!
Enter book title (or 'exit' to quit): Harry Potter
Book not available.
Enter book title (or 'exit' to quit): exit
```

Total searches made: 2

## 7. Supermarket Sales Tracker (3D List)

Concepts: 3D Lists · Indexing · Data Types · print

### Problem Statement:

- A supermarket chain has 2 branches, each with 3 product categories, each category tracking sales for 4 weeks.
- Store all weekly sales figures in a 3D list: [branch][category][week].
- Branch names: ['Shibuya', 'Shinjuku']
- Category names: ['Food', 'Drinks', 'Snacks']
- Print a full sales report: for each branch, show each category's 4-week sales.
- Calculate and print the total sales for each branch.
- Find and print which branch had the highest single-week sale and in which category.

### Acceptance Criteria:

- Define a 3D list with shape [2][3][4] (2 branches, 3 categories, 4 weeks).
- Use three levels of index access: sales[b][c][w].
- Sum all values in a branch using nested index access (no sum()).
- Track the highest single weekly sale and its branch/category using variables.
- Print branch and category names from their respective lists.

### Expected Output:

```
=====
SUPERMARKET SALES REPORT
=====

Branch: Shibuya
Food | Week 1: 120000 Week 2: 135000 Week 3: 118000 Week 4: 142000
Drinks | Week 1: 85000 Week 2: 90000 Week 3: 88000 Week 4: 95000
Snacks | Week 1: 45000 Week 2: 50000 Week 3: 47000 Week 4: 53000
Branch Total: 1068000

Branch: Shinjuku
Food | Week 1: 155000 Week 2: 160000 Week 3: 148000 Week 4: 170000
Drinks | Week 1: 92000 Week 2: 98000 Week 3: 94000 Week 4: 102000
Snacks | Week 1: 60000 Week 2: 65000 Week 3: 58000 Week 4: 70000
Branch Total: 1272000

Highest single-week sale: 170000
Branch: Shinjuku | Category: Food | Week 4
```

## 8. Gym Membership Fee Calculator

Concepts: While Loop · Conditionals · Data Types · print

### Problem Statement:

- Use a while loop to process multiple gym members.
- Each iteration: take member name, age (int), and plan (basic/premium/vip).
- Calculate monthly fee: basic=2000, premium=4000, vip=7000.
- Members aged 60 and above get a 20% senior discount on any plan.
- Members aged 15 and below get a 30% student discount on any plan.
- If the user enters 'stop' as the name, exit the loop.
- After loop, print total members processed and total revenue collected.

### Acceptance Criteria:

- Use while True with break on name == 'stop'.
- Use int() for age, .lower() for plan comparison.
- Use if/elif/else for plan fee selection.
- Apply senior/student discount using separate if/elif conditions.
- Track member count and total revenue with variables outside the loop.

### Expected Output:

```
Enter member name (or 'stop' to exit): Aiko
Enter age: 65
Enter plan (basic/premium/vip): premium
Aiko -> Plan: premium | Fee: 4000 | Discount: 20% | Payable: 3200.0
Enter member name (or 'stop' to exit): Kenji
Enter age: 14
Enter plan (basic/premium/vip): basic
Kenji -> Plan: basic | Fee: 2000 | Discount: 30% | Payable: 1400.0
Enter member name (or 'stop' to exit): stop
Total members: 2
Total revenue: 4600.0
```

---

## 9. Train Schedule & Seat Availability (2D List)

Concepts: 2D Lists · While Loop · Conditionals · Data Types

### Problem Statement:

- A railway system manages 4 trains, each with 5 seat classes.
- Each seat class stores the number of available seats.
- Store all data in a 2D list: [train][seat\_class].
- Train names: ['Nozomi', 'Hikari', 'Kodama', 'Sakura']
- Seat classes: ['Green', 'Reserved', 'Unreserved', 'Standing', 'Disabled']
- Print the full availability table for all trains.
- Use a while loop to let a user search by train index (0-3); enter -1 to stop.
- For the searched train, print each seat class and mark as 'Available' if seats > 0, else 'Full'.

### Acceptance Criteria:

- Define a 2D list with 4 rows (trains) and 5 columns (seat classes).
- Store train names and class names in separate lists.
- Use index access `seats[train][class]` for all lookups.
- Use `while True` with `break` when `input == -1`.
- Use `if/else` for Available/Full status.

### Expected Output:

```
=====
TRAIN SEAT AVAILABILITY
=====
Train | Green | Reserved | Unreserved | Standing | Disabled
-----
Nozomi | 12 | 45 | 0 | 0 | 3
Hikari | 0 | 10 | 22 | 15 | 2
Kodama | 5 | 0 | 30 | 40 | 0
Sakura | 8 | 20 | 18 | 0 | 5

Enter train number to check (0-3, or -1 to exit): 1
--- Hikari Seat Status ---
Green Car : Full
Reserved : Available (10 seats)
Unreserved : Available (22 seats)
Standing : Available (15 seats)
Disabled : Available (2 seats)

Enter train number to check (0-3, or -1 to exit): -1
Goodbye!
```

## 10. Hospital Patient Management System

Concepts: All — print · String Methods · Conditionals · Lists · While Loop · Data Types

### Problem Statement:

- Use a while loop to register patients until the user types 'done'.
- For each patient: take name, age (int), and severity level (1–10, int).
- Store all patient records in a 2D list: each row is `[name, age, severity]`.
- After registration, print all patients with their details.
- Apply priority rules:
  - Age  $\geq 60$  OR severity  $\geq 7 \rightarrow$  Priority Treatment
  - Age  $\geq 18$  AND severity  $\geq 4 \rightarrow$  Normal Treatment
  - Otherwise  $\rightarrow$  Standard Queue
- Print priority status next to each patient.
- Count and print how many patients fall in each priority category.

### Acceptance Criteria:

- Use while True with break on name == 'done'.
- Use .title() on patient name before storing.
- Use int() for age and severity.
- Store each patient as a list [name, age, severity] inside a 2D list.
- Use if/elif/else for priority classification.
- Use three counter variables for priority, normal, and standard counts.

### Expected Output:

```
Enter patient name (or 'done'): tanaka hiroshi
Enter age: 68
Enter severity (1-10): 5
Enter patient name (or 'done'): yuki sato
Enter age: 25
Enter severity (1-10): 8
Enter patient name (or 'done'): done
--- Patient Report ---
Tanaka Hiroshi | Age: 68 | Severity: 5 -> Priority Treatment
Yuki Sato | Age: 25 | Severity: 8 -> Priority Treatment
Priority Treatment : 2
Normal Treatment : 0
Standard Queue : 0
```

---